

---

# IndustryRAG-Bench: A RAG Benchmark for Company Internal Knowledge

---

Yuhong Sun<sup>1†</sup>   Joachim Rahmfeld<sup>1†</sup>   Mark H. Butler<sup>2</sup>   Weijia Chen   Roshan Desai<sup>1</sup>

<sup>1</sup>Onyx   <sup>2</sup>UC Berkeley

{yuhong, joachim}@onyx.app   mhbutler@berkeley.edu

## Abstract

Retrieval-Augmented Generation (RAG) has become the standard approach for grounding large language models in information that was not available during training. While existing datasets and benchmarks focus on web or other public sources, there is still no widely adopted dataset that realistically reflects the nature of company-internal knowledge. Meanwhile, startups, enterprises, and researchers are increasingly developing AI Agents designed to operate over exactly this kind of proprietary data. To help close this gap, we introduce a new dataset that simulates enterprise knowledge environments and provide a flexible framework for extending and tailoring the data to specific use cases.

We present **IndustryRAG-Bench**, a dataset consisting of approximately **500,000 synthetic documents** spanning nine enterprise source types (Slack, Gmail, Linear, Google Drive, HubSpot, Fireflies, GitHub, Jira, and Confluence) and **500 questions** across ten categories that test distinct retrieval and reasoning capabilities. The corpus is generated with cross-document coherence—grounded in shared projects, people, and initiatives—and augmented with realistic noise such as misfiled documents, near-duplicates, and conflicting information. The question set ranges from simple single-document lookups to multi-document reasoning, constrained retrieval, conflict resolution, and recognizing when information is absent. Alongside the dataset, we release the full generation framework, enabling teams to produce their own tailored datasets for different industries, company sizes, and source applications. The dataset, code, evaluation harness, and leaderboard are available at <https://github.com/onyx-dot-app/IndustryRAG-Bench>.

## 1 Introduction

The growing adoption of Retrieval-Augmented Generation (RAG) has driven a wave of benchmark development for evaluating retrieval and generation quality. Prominent datasets such as Natural Questions [Kwiatkowski et al., 2019], MS MARCO [Bajaj et al., 2018], and the BEIR meta-benchmark [Thakur et al., 2021] have been instrumental in advancing the field, while multi-hop reasoning benchmarks like HotpotQA [Yang et al., 2018], MuSiQue [Trivedi et al., 2022], and BrowseComp-Plus [Chen et al., 2025] test more complex retrieval chains. Domain-specific efforts have also emerged, including PubMedQA [Jin et al., 2019] for biomedical literature and FinQA [Chen et al., 2021] for financial reasoning. More recently, MTEB [Muennighoff et al., 2023], KILT [Petroni et al., 2021], and CRAG [Yang et al., 2024] have provided unified evaluation frameworks spanning multiple tasks.

Despite this progress, these benchmarks share a common assumption: the underlying corpus is drawn from publicly accessible sources—web pages, Wikipedia, scientific articles, forums, or curated

---

<sup>†</sup>Equal contribution.

Preprint. Under review.

Table 1: Enterprise source types and approximate document volumes.

| # | Source Type  | Approx. Volume | Description                                                     |
|---|--------------|----------------|-----------------------------------------------------------------|
| 1 | Slack        | 275,000        | Internal channels and team discussions                          |
| 2 | Gmail        | 120,000        | Email threads from management, sales, leadership, and ICs       |
| 3 | Linear       | 35,000         | Project management tickets for engineering, product, and design |
| 4 | Google Drive | 25,000         | Shared files and collaborative documents                        |
| 5 | HubSpot      | 15,000         | CRM records for sales pipeline and customer tracking            |
| 6 | Fireflies    | 10,000         | Meeting transcripts for internal and external-facing meetings   |
| 7 | GitHub       | 8,000          | Pull requests and review comments across code repositories      |
| 8 | Jira         | 6,000          | Support tickets, both internal and customer-facing              |
| 9 | Confluence   | 5,000          | Wikis, runbooks, and structured documentation                   |

Table 2: Question categories, counts, and targeted capabilities.

| #  | Category            | Count | Description                                                               |
|----|---------------------|-------|---------------------------------------------------------------------------|
| 1  | Basic               | 175   | Single ground truth document; moderate keyword overlap.                   |
| 2  | Semantic            | 125   | Roundabout phrasing with low keyword overlap; requires semantic matching. |
| 3  | Intra-Doc Reasoning | 40    | Combining information from distant sections of one long document.         |
| 4  | Project Related     | 40    | Aggregating knowledge from related documents within a single project.     |
| 5  | Constrained         | 30    | Qualifiers narrow multiple relevant documents to one correct answer.      |
| 6  | Conflicting Info    | 20    | Documents contradict each other; requires a complete, correct answer.     |
| 7  | Completeness        | 20    | Requires fetching all relevant documents exhaustively.                    |
| 8  | Miscellaneous       | 20    | Targets informal, off-topic, or loosely organized documents.              |
| 9  | High Level          | 10    | Synthesis across the corpus; no single gold document.                     |
| 10 | Info Not Found      | 20    | Answer is absent from the corpus; requires recognizing absence.           |

knowledge bases. Yet one of the most consequential applications of RAG today is over **company-internal knowledge**—an organization’s documents, messages, tickets, and records—and no widely adopted public benchmark reflects its characteristics. For example, KARLBench [Databricks, 2025] reflects growing industry interest in enterprise knowledge grounding, but its corpus is predominantly public documents with only a small unreleased proprietary subset.

Company-internal data differs from public corpuses in ways that existing benchmarks do not capture: it is heterogeneous across source types, dense with internal terminology, interconnected through shared projects and people, and noisy with misfiled, outdated, or conflicting information. To create a dataset that faithfully reflects these properties, we identify five design considerations:

1. **Cross-document coherence.** Documents should be connected through shared initiatives, people, and decisions, not generated in isolation.
2. **Realistic volume distribution.** The ratio of documents across source types and the topic distribution within each source should reflect real-world patterns.
3. **Realistic noise.** The dataset should include misfiled documents, near-duplicates, and conflicting or outdated information rather than assuming a clean knowledge base.
4. **Internal terminology.** Documents should use project codenames, product-specific acronyms, and organizational jargon that are essential for correct retrieval but meaningless outside the company.
5. **Generality across enterprise settings.** The generation framework must support diverse industries, company stages, and organizational structures rather than encoding assumptions specific to a single domain.

Beyond these design principles, there are also **pragmatic constraints** that any large-scale generation effort must address. Generating up to millions of documents with LLMs is expensive and time-consuming. Requiring full cross-document awareness for every document, while ideal for coherence, would make the generation process prohibitively costly. Our generation pipeline addresses this

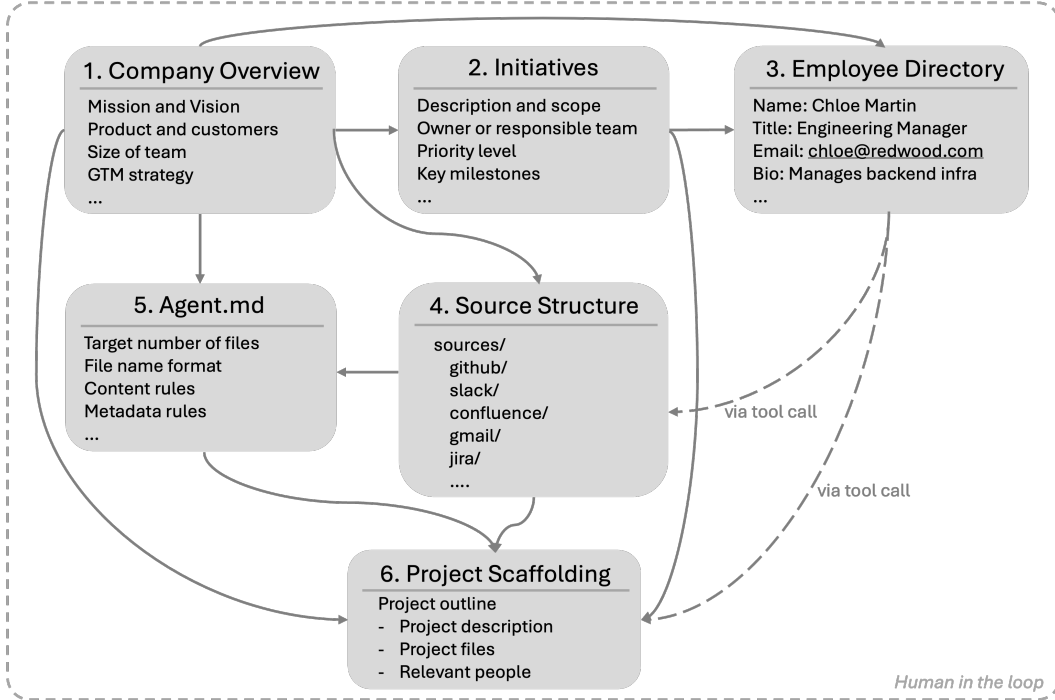


Figure 1: Overview of the generation framework scaffolding. Each downstream generation step is grounded in a subset of these five human-in-the-loop artifacts, ensuring cross-document coherence throughout the corpus.

tension by using targeted strategies: high-fidelity, context-rich generation for smaller clusters of tightly coupled documents, and cost-efficient, scaffolding-guided generation for the high-volume bulk of the corpus.

We release the dataset, generation framework, and a public leaderboard. The remainder of the paper is organized as follows: Section 2 gives an overview of the dataset, Section 3 describes the generation methodology, Section 4 covers the evaluation framework, Section 5 presents baseline experiments, Section 6 discusses limitations, and Section 7 concludes with future directions.

## 2 Dataset Overview

The IndustryRAG-Bench corpus simulates a technology company called “Redwood Inference” and is organized across the source types listed in Table 1. The question set spans ten categories (Table 2), each targeting a distinct retrieval or reasoning capability. The volume distribution is intentionally non-uniform: Slack and Gmail dominate the corpus by an order of magnitude over sources like Confluence and Jira, reflecting the reality that informal communication channels produce far more documents than curated knowledge bases.

Basic and Semantic questions test standard retrieval with varying levels of lexical overlap. Intra-Document Reasoning evaluates whether systems can synthesize information scattered across a long document rather than relying on a single passage. Project Related and Constrained questions introduce multi-document reasoning and precision filtering, respectively. Conflicting Info and Completeness push further by requiring systems to reconcile contradictions or achieve exhaustive recall. Miscellaneous questions target informal, off-topic, or ad-hoc documents stored in less predictable locations. High Level questions require broad synthesis where no single document suffices; multiple different subsets of documents may each produce a complete and correct answer. Info Not Found questions test whether the system can recognize when the corpus does not contain the answer, rather than hallucinating one from superficially related documents.

### 3 Data Generation Methodology

The generation pipeline consists of three stages: building the document set, introducing noise, and generating questions. Two requirements shape the pipeline beyond the dataset-level design principles. The corpus must be **question-aware**: certain question types—completeness, conflicting info, constrained—require specific document structures generated intentionally rather than discovered after the fact across hundreds of thousands of documents. And every question must have **verifiable ground truth**: gold documents and answers targeted enough for reliable evaluation without exhaustive manual review, paired with phrasing that avoids trivial keyword overlap that would let retrieval systems saturate the benchmark.

#### 3.1 Building the Document Set

Document generation proceeds in three phases: scaffolding, high fidelity generation, and high volume generation.

**Scaffolding.** All generation begins from five top-level artifacts produced through human-in-the-loop collaboration with an LLM:

1. A **company overview** covering mission, products, technology, business model, competitive positioning, team size, and funding history.
2. A set of **high-level initiatives** describing the company’s major strategic efforts.
3. An **employee directory** with names, roles, departments, and reporting relationships.
4. A **source structure** defining the directory hierarchy for each of the nine source types, informed by the company context and real-world organizational patterns.
5. **agents.md files** placed throughout the directory tree to specify, in natural language, the expected format and content of documents in each location (e.g., that GitHub contains only pull requests with review comments, not issues).

These artifacts are referenced by nearly every downstream generation step and ensure that documents share a common organizational reality. The scaffolding extends to the project level: initiatives are decomposed into projects, each targeting roughly 100 documents, and individually enriched with detailed descriptions, document manifests, and assigned employees. Appendix B.1 provides procedural details for the project-level decomposition.

**High-fidelity generation.** The core document set is produced with maximal cross-document awareness. Each document is generated with access to the company overview, its project’s full manifest of sibling documents, the `agents.md` files along its path, and a read tool for inspecting previously generated project documents on demand. This ensures that documents within a project exhibit the implicit cross-references—shared decisions, referenced meetings, linked tickets—that characterize real enterprise knowledge. A separate step produces **completeness clusters**: groups of 4–10 documents generated with full mutual visibility and facts deliberately distributed across the group, supporting questions that require exhaustive recall.

**High-volume generation.** The bulk of the corpus is produced through a cost-efficient pipeline that trades per-document context for scale. The central challenge is **model drift**: without tight steering, the LLM converges on a narrow set of themes and produces near-duplicate documents. In a controlled experiment, we prompted the model to generate 100 documents using only the company overview for context. The model judged that over 40% had a very close sibling, a pattern consistent across all tested LLMs and temperature settings. To counteract this, we introduce a **topic scaffolding** layer that hierarchically decomposes each source type into topics and subtopics, with document counts calibrated to real-world distributions. Topics are recursively split until each leaf contains at most 500 documents. During generation, the model sees only the company overview, key initiatives, the relevant `agents.md`, and file paths within the same leaf topic. This narrow context window substantially reduces cost while eliminating the runaway duplication observed without scaffolding. Appendix B provides full procedural details for all three phases.

Table 3: Noise types introduced after base document generation.

| Noise Type          | Mechanism                                                                                                                       |
|---------------------|---------------------------------------------------------------------------------------------------------------------------------|
| Random shuffle      | Documents relocated within their source type via directory-walk sampling                                                        |
| LLM-based shuffle   | Documents relocated to structurally plausible but incorrect locations                                                           |
| Near-duplicates     | New version generated with specific facts changed; may cross source types                                                       |
| Miscellaneous files | Informal, off-topic content in ad-hoc directories (e.g., <code>slack/memes</code> , <code>google_drive/.../misc-assets</code> ) |

Table 4: Question generation families and their corpus-awareness approaches.

| Family              | Question Types                                           | Approach                                                                                        |
|---------------------|----------------------------------------------------------|-------------------------------------------------------------------------------------------------|
| Single-document     | Basic, Semantic, Intra-Doc Reasoning, Miscellaneous      | Sample one document; generate a question with type-specific phrasing and validation constraints |
| Corpus-exploration  | Project Related, Constrained, High Level, Info Not Found | LLM agent explores the corpus with file-system tools before formulating a question              |
| Structure-dependent | Conflicting Info, Completeness                           | Built from document structures created in Sections 3.1 and 3.2                                  |

### 3.2 Adding Noise

Real enterprise corpuses are not clean. After the base document set is generated, we introduce four types of noise to approximate real-world imperfections (Table 3).

Random shuffling preserves format consistency by staying within a source type, while LLM-based shuffling produces more realistic misfiling patterns—documents end up in adjacent directories or nearby branches of the hierarchy rather than arbitrary locations. Near-duplicate pairs introduce controlled factual divergence and are tracked for later use in conflicting-info questions. Miscellaneous files support a question category targeting informal content in less predictable locations. Procedural details for each noise type are provided in Appendix B.

### 3.3 Generating Questions

The ten question types are designed to reflect the richness and complexity of realistic questions in enterprise settings. For example, projects that receive multiple updates over their lifetime give rise to questions about conflicting or outdated information, which is rarely emphasized in public corpuses. Each type is produced through a distinct generation flow, and all flows share a common output structure: a question, a gold answer, a list of atomic answer facts for fine-grained completeness scoring, and (where applicable) gold document identifiers. The flows fall into three families based on their corpus-awareness requirements (Table 4).

Single-document flows differ primarily in sampling criteria and prompt steering. For instance, semantic questions require roundabout phrasing with minimal lexical overlap, while intra-doc reasoning questions must link information from distant sections and are rejected if answerable from a single passage. Corpus-exploration flows equip the generation model with tools (`glob`, `grep`, `read`) to discover document relationships before proposing a question. Constrained questions additionally produce **anti-hallucination facts**: negative statements that catch errors a system might make if it retrieves a distractor document instead of the gold document. Structure-dependent flows leverage the near-duplicate pairs (Section 3.2) and completeness clusters (Section 3.1) created during earlier stages, ensuring the document structures needed for these question types exist by construction. Detailed per-type generation flows are provided in Appendix C.

### 3.4 Quality Assurance

Generating questions from documents does not guarantee that the source document is the only, or even the best, document for that question. A question back-generated from one document may be equally or better answered by an unrelated document elsewhere in the corpus. Similarly, while project-level scaffolding enforces coherence within a project, a document produced during high-volume generation may coincidentally contain information that supplements or supersedes a project-based gold answer.

At the scale of 500,000 documents, these collisions are not hypothetical edge cases but expected outcomes. Three mechanisms address this inherent uncertainty.

**LLM-based conformance checks.** Each question type includes a validation step that verifies type-specific requirements: for example, that intra-document reasoning questions cannot be answered from a single contiguous passage, or that high-level questions are not answerable from any one document. Questions failing validation are regenerated or discarded.

**Multi-retriever pooling.** Three independent retrieval systems, included in the released code, are run against the question set: a BM25 keyword search (OpenSearch), a vector search (OpenAI text-embedding-3-large embeddings with cosine similarity over Qdrant), and a bash agent with shell tools. Their retrieved documents are pooled with the original gold set and evaluated through the three-judge consensus process described in Section 4.3 to identify missing gold documents or flag incorrect ones. This pooling improves gold set coverage beyond what any single generation or retrieval pass can achieve, though it does not guarantee exhaustive correctness. Not all corrections are recoverable: for example, if pooling reveals a second gold document for a question type that requires exactly one, the question must be discarded rather than corrected, since updating the gold set would violate the type’s structural constraints. To account for this loss, each question type was initially overgenerated beyond its target count. After conformance checks and retriever-based corrections, questions that received corrections were discarded and the remaining questions were randomly sampled down to the desired volume per category.

**Ongoing human curation.** It is more feasible to verify that a newly proposed document is better than the current gold document than to confirm exhaustively that no better match exists anywhere in the corpus. As benchmark submissions are collected, we will also collect corrections to gold documents and answers surfaced by participating systems. These corrections will be reviewed and periodically incorporated into updated releases of the dataset, allowing the ground truth to improve over time as more systems exercise different retrieval strategies against the dataset. We encourage the community to participate through either submissions to the leaderboard<sup>1</sup> or via issues on GitHub.<sup>2</sup>

## 4 Evaluation Framework

Evaluating RAG systems over enterprise corpuses introduces challenges beyond standard retrieval benchmarks. Gold document sets are inherently incomplete: at the scale of 500,000 documents, it is infeasible to guarantee that no unlabeled document provides a better answer than the annotated gold set. The evaluation framework must therefore treat gold labels as revisable hypotheses rather than fixed ground truth, while still producing stable, interpretable metrics. We address this through a correction-aware scoring pipeline (Section 4.3) that updates gold labels when evidence warrants it, combined with metrics that separately measure answer quality and retrieval performance.

### 4.1 Metrics

Each question is scored along four dimensions: correctness, completeness, document recall, and invalid extra documents.

**Correctness** is a binary LLM-based judgment of whether the candidate answer is broadly aligned with the gold answer. The judge is lenient toward stylistic differences, additional context, and supplementary detail, but the answer must not contain factual conflicts with the gold answer or mismatched quantities.

**Completeness** measures the fraction of atomic answer facts supported by the candidate answer. Each fact from the gold `answer_facts` list is independently validated by an LLM judge that checks whether the candidate answer contains or implies that fact. The score is the fraction of facts validated. Because each fact is evaluated in isolation, no single judgment propagates error to others, and the metric provides fine-grained signal about which aspects of the answer are missing.

---

<sup>1</sup><https://huggingface.co/spaces/onyx-dot-app/IndustryRAG-Bench-Leaderboard>

<sup>2</sup><https://github.com/onyx-dot-app/IndustryRAG-Bench>

**Document Recall** is the fraction of gold documents that appear in the candidate’s retrieved set, computed only for question types that carry expected document identifiers.

**Invalid Extra Documents** counts retrieved documents that are neither in the gold set nor classified as “valid” (relevant but not required) by the evaluation judges, again restricted to question types with expected documents. This metric captures retrieval noise: systems that retrieve broadly may achieve high recall at the cost of surfacing irrelevant material.

Correctness and completeness are evaluated independently: the correctness judge has no visibility into the answer facts, and fact validation has no visibility into the correctness judgment. Prior to scoring, citations are stripped from candidate answers to prevent the judge from being biased by reference formatting rather than substantive content.

**Leaderboard.** The leaderboard ranks systems by a single aggregate score. For each question, the score is the completeness percentage if the answer is correct and zero otherwise. The overall score is the average across all questions.

## 4.2 Comparative Evaluation

For head-to-head system comparison, the framework supports a comparative mode that evaluates two systems on the same question set. Retrieved documents from both systems are pooled through the same three-judge correction process (Section 4.3), and both systems are scored against the resulting (potentially updated) gold set. A separate three-judge panel produces a per-question preference between the two systems’ answers; to mitigate positional bias, system ordering is randomly swapped before presentation and mapped back afterward. Retrieved documents are partitioned into shared and system-unique subsets to support analysis of where systems diverge.

## 4.3 Gold Set Correction

*Note: All leaderboard submissions are scored against the same fixed gold set—the one included in the initial release—so results are directly comparable regardless of when a system is evaluated. Corrections are accumulated offline and incorporated into future versioned releases; no retroactive rescoring occurs within a release version.*

A key design decision is that the gold set is not static. When a system retrieves documents that differ from the annotated gold set, the evaluation pipeline determines whether the gold labels should be updated before scoring. The correction procedure operates as follows.

The union of gold documents and candidate-retrieved documents is evaluated through a **three-judge consensus** process. Because document relevance can be ambiguous and because individual LLM judgments exhibit variance, three independent judges each classify every document as “required” (essential to answering the query), “valid” (relevant but not necessary), or “invalid” (does not help answer the query). An ordinal majority vote determines the final classification, with gold-biased tie-breaking: gold documents are kept as “required” unless a majority votes them “invalid,” while candidate documents need a strict majority to be promoted. If any single run produces a classification consistent with the original gold set, the pipeline short-circuits and retains the original labels.

Only “required” documents enter the gold set. “Valid” documents are recorded but excluded from the gold set and are not penalized as invalid extra documents during scoring. This prevents near-duplicates and corroborating sources from inflating the invalid count. When the required set changes, the gold answer and answer facts are regenerated from the updated documents, preserving **anti-hallucination facts** from the original fact set (negative statements designed to catch specific errors). Corrected questions are flagged in the output.

This correction mechanism reflects the reality that gold set quality can improve incrementally as more systems exercise different retrieval strategies against the corpus. It is more feasible to verify that a newly proposed document is relevant than to confirm exhaustively that no better match exists anywhere in the corpus. Over time, as benchmark submissions accumulate, the ground truth converges toward more complete and accurate labels.

Table 5: Semantic similarity characteristics across datasets. † BrowseComp-Plus lacks separate source types; values shown are in-question and cross-question similarities as approximations.

| Metric                       | IndustryRAG (511K) | Onyx Internal (12K) | BrowseComp-Plus (70K) |
|------------------------------|--------------------|---------------------|-----------------------|
| Avg. intra-source similarity | 0.44               | 0.37                | 0.23 <sup>†</sup>     |
| Avg. cross-source similarity | 0.37               | 0.27                | 0.14 <sup>†</sup>     |
| Avg. top-10 NN similarity    | 0.75               | 0.68                | 0.55                  |
| Avg. random pair similarity  | 0.38               | 0.27                | 0.14                  |

## 5 Experiments and Analysis

### 5.1 Dataset Analysis

To verify that the generated corpus credibly reflects the properties of real enterprise data, we compare it against both a real enterprise dataset and a publicly sourced RAG benchmark. We focus on how documents relate to each other within and across source types, and how densely they cluster in embedding space. We measure three quantities to characterize this structure:

1. **Intra-source similarity.** Mean pairwise cosine similarity between randomly sampled document embeddings from the same source type. Higher values indicate a more topically focused source.
2. **Cross-source similarity.** Mean pairwise cosine similarity between randomly sampled document embeddings from different source types. Higher values indicate greater topical overlap between sources.
3. **Local neighborhood similarity.** Mean cosine similarity of each document’s 10 nearest neighbors. Higher values indicate that documents are more closely surrounded by related content, increasing retrieval difficulty.

In realistic enterprise data, we expect all three metrics to be relatively high: a company’s documents revolve around its products, projects, and operations, so even documents from different sources discuss related topics. Intra-source similarity should exceed cross-source similarity, but not by a wide margin, since different sources (Slack, Confluence, Jira) serve different purposes but cover the same business. By contrast, a publicly sourced benchmark spanning unrelated domains should show substantially lower values across all three metrics.

We compare three datasets: the IndustryRAG-Bench corpus, Onyx’s own internal corpus, and BrowseComp-Plus [Chen et al., 2025], a publicly sourced benchmark. BrowseComp-Plus lacks separate source types, so intra-source and cross-source comparisons are not available; instead, we report in-question and cross-question similarities. All documents are embedded with Cohere embed-v4.0 using a chunk size large enough that nearly all documents are represented as a single vector.

Table 5 confirms the expected patterns. IndustryRAG-Bench closely tracks the real enterprise dataset across all metrics, with both showing substantially higher similarities than BrowseComp-Plus. The intra-source similarities (0.44 vs. 0.37) and cross-source similarities (0.37 vs. 0.27) for IndustryRAG-Bench are higher than the Onyx internal data. This is expected: LLM-generated content tends to stay on-topic and struggles to reproduce the peripheral, loosely related material that accumulates in real enterprise environments. The gap between intra-source and cross-source similarity is small in both enterprise datasets, reflecting the expected property that different sources discuss related topics despite differing in format and purpose.

Running the same analysis on a subset of approximately one-fifth of the corpus produced no significant differences in any of the metrics, suggesting that these properties are stable across scale.

The local neighborhood similarity (0.75 for IndustryRAG-Bench, 0.68 for Onyx, 0.55 for BrowseComp-Plus) reflects how many near-miss documents surround a typical query target. In enterprise data, documents share terminology, projects, and context, producing a high concentration of plausible candidates regardless of retrieval method. This makes distinguishing the correct document from its neighbors harder than in a benchmark where topics are largely unrelated.



Table 6: Overall performance of baseline retrieval systems. Best result per metric in **bold**. † Vector uses OpenAI text-embedding-3-large. ‡ Bash Agent uses GPT-5.4 with low reasoning.

| Metric              | BM25        | Vector <sup>†</sup> | Bash Agent <sup>‡</sup> |
|---------------------|-------------|---------------------|-------------------------|
| Correctness (%)     | <b>68.8</b> | 51.4                | 60.6                    |
| Completeness (%)    | 56.0        | 42.9                | <b>61.1</b>             |
| Document Recall (%) | <b>68.4</b> | 46.0                | 55.8                    |
| Invalid Extra Docs  | 9.0         | 9.3                 | <b>2.0</b>              |

*Note:* Correctness and completeness can exceed document recall because (1) non-gold documents may contain overlapping facts, (2) partial retrieval can still produce a correct answer, and (3) some facts are corroborated across multiple gold documents.

Table 7: Per-type correctness (%) and document recall (%) for baseline systems. Best result per row in **bold**. High Level and Info Not Found carry no gold documents, so recall is omitted (—). † Vector uses OpenAI text-embedding-3-large. ‡ Bash Agent uses GPT-5.4 with low reasoning.

| Question Type            | BM25         |             | Vector <sup>†</sup> |        | Bash Agent <sup>‡</sup> |              |
|--------------------------|--------------|-------------|---------------------|--------|-------------------------|--------------|
|                          | Corr.        | Recall      | Corr.               | Recall | Corr.                   | Recall       |
| Basic (175)              | <b>79.4</b>  | <b>77.7</b> | 50.3                | 48.6   | 65.7                    | 61.7         |
| Semantic (125)           | <b>44.8</b>  | <b>43.2</b> | 32.8                | 24.8   | 42.4                    | 29.6         |
| Intra-Doc Reasoning (40) | <b>85.0</b>  | <b>90.0</b> | 57.5                | 57.5   | 75.0                    | 75.0         |
| Project Related (40)     | <b>60.0</b>  | <b>65.5</b> | 52.5                | 49.2   | 37.5                    | 43.2         |
| Constrained (30)         | <b>76.7</b>  | <b>85.0</b> | <b>76.7</b>         | 83.3   | <b>76.7</b>             | 78.3         |
| Conflicting Info (20)    | <b>90.0</b>  | <b>82.5</b> | 60.0                | 55.0   | 80.0                    | 72.5         |
| Completeness (20)        | <b>40.0</b>  | 46.5        | <b>40.0</b>         | 33.4   | 35.0                    | <b>59.0</b>  |
| Miscellaneous (20)       | 85.0         | 90.0        | 75.0                | 75.0   | <b>90.0</b>             | <b>100.0</b> |
| High Level (10)          | 50.0         | —           | <b>60.0</b>         | —      | <b>60.0</b>             | —            |
| Info Not Found (20)      | <b>100.0</b> | —           | <b>100.0</b>        | —      | <b>100.0</b>            | —            |

## 5.2 Baseline Systems Performance

Three retrieval configurations are evaluated against the full question set, representing the major families of retrieval approaches. **BM25** [Robertson and Zaragoza, 2009] and **vector search** are established baselines for keyword and semantic retrieval, respectively. BM25 uses OpenSearch with a standard analyzer over a single concatenated text field. Vector search embeds documents with OpenAI’s text-embedding-3-large [OpenAI, 2024] (3072 dimensions) and retrieves via cosine similarity over Qdrant. Both retrieve a fixed top-10 document set per query. **Bash agent** retrieval represents a more recent approach following the agentic keyword search paradigm [Subramanian et al., 2025], an approach seeing significant industry adoption.<sup>3</sup> The agent is equipped with shell tools (grep, find, head, etc.) to iteratively explore the corpus directory structure, selecting a variable number of documents per question with a 10-minute time budget. The Bash Agent was evaluated with low and medium reasoning with no significant difference in the top-level metrics; the low reasoning numbers are reported in Table 6. All systems use GPT-5.4 (Medium Reasoning) for answer generation and answer evaluation to provide a fair comparison.

BM25 leads on correctness and document recall. Vector search underperforms expectations even on semantic questions (32.8% correctness, 24.8% recall), the category explicitly designed to favor embedding-based retrieval by suppressing keyword overlap. One possible explanation is that embedding models are trained on public corpuses (web pages, Wikipedia, forums) and have limited exposure to enterprise-specific vocabulary such as project codenames and internal acronyms, structured formats like tickets and CRM records, and conversational styles typical of internal messaging. This merits further investigation, which we leave to future work. It does, however, reinforce the premise motivating IndustryRAG-Bench: public benchmark performance does not necessarily transfer to enterprise retrieval.

The Bash Agent is the only system to exceed 60% overall completeness, achieving the highest score on that metric (61.1%) despite trailing BM25 on correctness and recall overall. Its strength is iterative

<sup>3</sup><https://vercel.com/changelog/introducing-bash-tool-for-filesystem-based-context-retrieval>

discovery: once it reaches a relevant document, it can explore nearby files and similar patterns to find related documents. This possibly explains the high recall on *completeness*-type questions (59.0% vs. 46.5% for BM25 and 33.4% for vector), the hardest category overall. It is also the only system to achieve perfect recall on miscellaneous questions, highlighting its strength at retrieving documents outside of dense topic areas. However, the Bash Agent approach comes at a substantial cost in latency and LLM cost: each question may take up to 10 minutes of iterative exploration, compared to a fraction of a second for retrieval plus a few seconds for generation with either BM25 or vector search.

## 6 Limitations

The IndustryRAG-Bench dataset is synthetic: no real enterprise corpus of comparable scale and diversity can be released publicly without compromising confidentiality. While the generation framework addresses many of the resulting challenges, several limitations remain inherent to LLM-based corpus construction at this scale. We group them into generation fidelity, representation constraints, design tradeoffs, and evaluation.

**Generation fidelity.** Four properties of LLM-based generation reduce the realism of individual documents. First, *long-tail errors* are statistically expected rather than edge cases at this scale: the initial corpus contained documents with control characters in file names, malformed metadata, and other artifacts that required manual removal. Second, *synthetic randomness artifacts* arise because LLMs fall back on recognizable patterns when generating values that should appear arbitrary: Unix timestamps converge on round numbers like 123456789, company names default to “ACME” variants, and numerical distributions lack the irregularity of real data. These artifacts are cosmetic rather than structural, but could provide unintended retrieval signals. Third, *document structure and path drift* occurs despite guidance from agents.md files: the LLM does not always follow the specified layout, and natural-language-driven generation offers no strict schema enforcement. The same drift applies to file paths: some documents end up in locations that break the expected hierarchy (e.g., Slack messages placed directly under the top-level directory rather than inside a channel). At the scale of the released dataset these deviations are a minority, but they are non-negligible. Fourth, *LLM-generated conversations are unrealistically focused*: real Slack threads feature off-topic tangents, misunderstandings, and unrelated interjections that LLM-generated threads rarely reproduce. This affects retrieval difficulty in both directions: there are fewer trivially irrelevant documents to discard, but also less realistic noise for systems to filter through.

**Representation constraints.** All documents are stored as flattened JSON key-value pairs to standardize processing and export. This does not capture the nested structures, embedded objects, or rich formatting that real enterprise documents naturally exhibit: email threads with quoted reply trees, CRM records with nested contact hierarchies, or wiki pages with embedded tables and media.

**Design tradeoffs.** Three deliberate tradeoffs constrain the dataset’s scope. First, the high-volume generation flow restricts per-document context to keep costs manageable, with no access to the employee directory and only shallow cross-references scoped to leaf topics. Many high-volume documents consequently reference fictional contributors not grounded in the organizational chart, and cross-document coherence is weaker than in the high-fidelity core. This is benign for the released question set, which draws its multi-document and people-dependent questions exclusively from the high-fidelity portion, but it limits the kinds of questions that can be reliably generated against the bulk of the corpus. Second, the dataset simulates a single technology company at a specific stage and scale. Real enterprises vary widely across industries, organizational maturity, communication culture, and tooling choices, and the released corpus will be more representative of some organizations than others. Third, because several generation steps feed accumulated context back into the prompt, generation quality may degrade at scales substantially beyond the released corpus. The current methodology has not been stress-tested at larger volumes.

**Evaluation.** At 500,000 documents, exhaustive annotation is infeasible. Despite multi-retriever pooling and LLM-based conformance checks, the gold document sets and answers are revisable hypotheses rather than fixed ground truth, as discussed in Section 4. The correction mechanism addresses this incrementally, but any individual question’s gold labels may be incomplete or improvable, a property shared with virtually all large-scale retrieval benchmarks. We plan to release periodic

updates that incorporate corrections surfaced by benchmark participants, so that gold labels improve over time as more systems exercise different retrieval strategies against the corpus.

## 7 Conclusion and Future Work

We present IndustryRAG-Bench, a benchmark for evaluating RAG systems over company-internal knowledge. Our contribution consists of a 500,000-document synthetic corpus spanning nine enterprise source types, 500 questions across ten categories, and a correction-aware evaluation framework. We release the dataset, generation framework, evaluation harness, and a public leaderboard.

The generation framework supports adaptation to different industries, company sizes, and source applications without changes to the core methodology, enabling teams to produce tailored enterprise benchmarks rather than relying only on the released corpus.

Several directions for future question types remain open:

- **High-volume aggregation.** Questions requiring a large fraction of the corpus to stress-test exhaustive lookup at scale.
- **Recency-aware retrieval.** Questions asking for the latest document matching loosely specified criteria, combining topical relevance with temporal constraints.
- **People-centric questions.** Questions about employee responsibilities, expertise, and relationships. People are referenced inconsistently across sources (full names, handles, titles) and are rarely the primary subject of any single document.
- **True multi-hop reasoning.** Questions where the answer to one retrieval step reveals what to search for next, requiring sequential discovery across documents.

The benchmark is designed to improve over time. The leaderboard uses the initial released gold set to ensure consistent comparison; accumulated corrections from participating systems will be incorporated into future versioned releases. We invite the community to submit systems and contribute corrections that strengthen the ground truth for future participants.

## References

- Payal Bajaj, Daniel Campos, Nick Craswell, Li Deng, Jianfeng Gao, Xiaodong Liu, Rangan Majumder, Andrew McNamara, Bhaskar Mitra, Tri Nguyen, et al. MS MARCO: A human generated machine reading comprehension dataset. *arXiv preprint arXiv:1611.09268*, 2018.
- Zhiyu Chen, Wenhu Chen, Charese Smiley, Sameena Shah, Iana Borber, and Michael Bendersky. FinQA: A dataset of numerical reasoning over financial data. In *Proceedings of EMNLP*, 2021.
- Zhiyu Chen et al. BrowseComp-Plus: A controlled evaluation framework for browsing agents. *arXiv preprint arXiv:2508.06600*, 2025.
- Databricks. Meet KARL: A faster agent for enterprise knowledge, powered by custom RL. Technical report, Databricks, 2025.
- Qiao Jin, Bhuwan Dhingra, Zhengping Liu, William Cohen, and Xinghua Lu. PubMedQA: A dataset for biomedical research question answering. In *Proceedings of EMNLP*, 2019.
- Tom Kwiatkowski, Jennimaria Palomaki, Olivia Redfield, Michael Collins, Ankur Parikh, Chris Alberti, Danielle Epstein, Illia Polosukhin, Jacob Devlin, Kenton Lee, et al. Natural questions: A benchmark for question answering research. *Transactions of the Association for Computational Linguistics*, 7:453–466, 2019.
- Niklas Muennighoff, Nouamane Tazi, Loïc Magne, and Nils Reimers. MTEB: Massive text embedding benchmark. In *Proceedings of EACL*, 2023.
- OpenAI. New embedding models and API updates. OpenAI Blog, 2024.
- Fabio Petroni, Aleksandra Piktus, Angela Fan, Patrick Lewis, Yacine Yaber, et al. KILT: A benchmark for knowledge intensive language tasks. In *Proceedings of NAACL*, 2021.

- Stephen Robertson and Hugo Zaragoza. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389, 2009.
- Sathya Subramanian et al. Keyword search is all you need. *arXiv preprint arXiv:2602.23368*, 2025.
- Nandan Thakur, Nils Reimers, Andreas Rücklé, Abhishek Srivastava, and Iryna Gurevych. BEIR: A heterogeneous benchmark for zero-shot evaluation of information retrieval models. In *Proceedings of NeurIPS*, 2021.
- Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. MuSiQue: Multihop questions via single hop question composition. *Transactions of the Association for Computational Linguistics*, 10:539–554, 2022.
- Xiao Yang, Kai Sun, Hao Xin, Yushi Sun, Nikita Bhalla, Xiangsen Chen, Sajal Choudhary, Rongze Daniel Gui, Ziran Will Jiang, Ziyu Jiang, et al. CRAG – comprehensive RAG benchmark. In *Proceedings of NeurIPS (Datasets and Benchmarks Track)*, 2024.
- Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William Cohen, Ruslan Salakhutdinov, and Christopher D Manning. HotpotQA: A dataset for diverse, explainable multi-hop question answering. In *Proceedings of EMNLP*, 2018.

# Appendices

## A Repository and Code Guide

The generation framework and evaluation harness are available in the repository at <https://github.com/onyx-dot-app/IndustryRAG-Bench>. The dataset itself is distributed as release artifacts (not checked into the repository) and can be downloaded from the latest GitHub release.<sup>4</sup>

### A.1 Dataset Artifacts

The release includes the following artifacts:

- `all_documents.zip` — The full document corpus as a single archive.
- Per-source slices (`<source_type>_slice_<N>.zip`) — Individual archives of up to 5,000 documents each, for partial downloads.
- `questions.jsonl` — The 500 benchmark questions. Also available at the repository root.
- `extra_questions.jsonl` — 100 additional metadata-dependent questions excluded from the core benchmark.

### A.2 Evaluation Harness

Retrieval systems are expected to produce an answers file in JSONL format where each line contains a `question_id`, an `answer`, and a list of `document_ids` for the retrieved documents.

```
{"question_id": "qst_0001", "answer": "Your answer text...",  
 "document_ids": ["dsid_abc", "dsid_def"]}
```

**Metrics-based evaluation** scores a single system on the four metrics described in Section 4.1:

```
python -m src.scripts.answer_evaluation.metrics_based_eval \  
--answers-file answer_evaluation/answers.jsonl
```

**Comparative evaluation** scores two systems head-to-head with the three-judge consensus process:

```
python -m src.scripts.answer_evaluation.comparative_eval \  
--answer-file-1 answer_evaluation/system_1.jsonl \  
--answer-file-2 answer_evaluation/system_2.jsonl
```

Questions that receive gold set corrections during evaluation are flagged in the results and written to a separate updated questions file.

### A.3 Generation Pipeline

The generation framework lives under `src/` and is organized into four stages, each containing numbered scripts that are run sequentially (Table 8).

All scripts are resumable: they check for existing output and prompt before regenerating. Intermediate state is persisted in `generation_cache/`, allowing long-running stages to be interrupted and resumed. The repository's `quickstart.md` provides full invocation commands with recommended flags for every step.

## B Document and Noise Generation Details

### B.1 Project-Level Scaffolding

High-level initiatives are decomposed into projects—smaller efforts spanning engineering, go-to-market, operations, and internal functions—each targeting roughly 100 documents. Projects are

<sup>4</sup><https://github.com/onyx-dot-app/IndustryRAG-Bench/releases/latest>

Table 8: Generation pipeline stages and their purpose.

| Stage | Path                             | Scripts | Purpose                                                                       |
|-------|----------------------------------|---------|-------------------------------------------------------------------------------|
| 1     | <code>data_gen_stage_1...</code> | 1–9     | Scaffolding, high-fidelity documents, completeness clusters, volume documents |
| 2     | <code>data_gen_stage_2...</code> | 1–4     | Random shuffle, LLM-based shuffle, miscellaneous files, near-duplicates       |
| 3     | <code>data_gen_stage_3...</code> | 1–11    | Ten question types plus optional metadata questions                           |
| 4     | <code>data_gen_stage_4...</code> | 1       | Export and packaging                                                          |

individually enriched: the LLM generates a detailed description and a manifest of documents to produce, using tool access (`glob`, `tree`) to inspect the existing directory structure and `agents.md` files so that proposed documents respect source-type constraints. Each project is further annotated with relevant employees from the directory, including their roles within the project.

## B.2 High-Fidelity Generation

For each project, every document is generated with access to: (a) its own file name, path, and description; (b) the company overview; (c) the full enriched project description including the manifest of all sibling documents; (d) the `agents.md` files along its path; and (e) a read tool that allows the model to inspect previously generated project documents when additional detail is needed.

Completeness clusters are generated through a separate step that produces small groups of 4–10 documents where every document in the cluster is loaded into the model’s context simultaneously. Unlike the project-level generation, which operates over  $\sim 100$  documents and pulls in siblings on demand, completeness clusters guarantee full mutual visibility. Each cluster is anchored to a target question so that the generation knows which facts must be distributed across the group, ensuring that no single document contains the complete answer.

## B.3 High-Volume Generation and Topic Scaffolding

For each source type, the system generates a hierarchical decomposition of topics and subtopics, with estimated document counts per topic calibrated to match expected real-world distributions (e.g., an engineering Slack workspace should cover the full range of discussions one would actually expect, not just the handful of themes an LLM gravitates toward). Topics are recursively split until each leaf contains at most 500 documents. During generation, the model sees only the company overview, key initiatives, the `agents.md` for the target source type, and the file paths of other documents within the same leaf topic. This narrow context window, compared to the full project scaffolding used in high-fidelity generation, substantially reduces cost per document while the leaf-level path visibility nudges the model to produce complementary rather than overlapping content.

This design involves deliberate tradeoffs. Documents in the high-volume flow are not grounded in the employee directory, so they reference fictional contributors not present in the organizational chart. Cross-references are shallow and scoped to leaf topics rather than enforced globally. These limitations are benign for the released question set, which draws its multi-document and people-dependent questions from the high-fidelity core.

## B.4 Noise Generation Procedures

**Random shuffle.** A specified percentage of documents (5% in the released dataset) are randomly relocated within their source type. Documents are selected via a random walk over the directory tree so that selection reflects directory structure rather than being skewed by per-folder file counts. Cross-source shuffling is excluded to preserve format consistency: a ticketing system document with fields like “assignee” and “closed date” would be incoherent if placed in a Slack channel.

**LLM-based shuffle.** Real misfiling is not uniformly random; documents tend to end up in structurally plausible but incorrect locations: adjacent directories, parent/child folders, or nearby branches of the hierarchy. An additional 3% of documents (excluding those already shuffled) are relocated using an LLM that receives the document’s original path, its contents, and the source’s directory struc-

ture, and proposes a destination that is reasonable but suboptimal. Cumulatively, 8% of documents in the released dataset are affected by some shuffle operation.

**Near-duplicates with conflicting facts.** We sample documents using the directory-walk procedure described above and generate a near-duplicate at a new location with specific facts changed. Unlike the shuffle operations, near-duplicates may cross source-type boundaries: a Slack conversation might contain an update on information originally recorded in a CRM record. The generation model receives the original document’s path and contents along with the `agents.md` files at the new location, producing a document that is structurally appropriate for its destination while introducing controlled factual divergence from the original. These pairs are tracked in a generation cache and later used to produce conflicting-info questions.

**Miscellaneous directories and files.** Real archives contain informal discussion, work-in-progress drafts, and ad-hoc notes in poorly organized locations. We introduce miscellaneous directories (e.g., `slack/memes`, `google_drive/.../misc-assets`, `github/hackathons`) through a human-in-the-loop process: given the existing directory structure, the LLM proposes candidate directories that the user reviews and finalizes. These directories are populated with informal, off-topic documents generated with awareness of the company overview, relevant `agents.md` files, and previously generated miscellaneous files (to avoid the same topic-clustering problem that motivated the scaffolding approach). The volume of miscellaneous files is intentionally small: they exist not to shift the corpus distribution but to support a question category that targets informal content stored in less predictable locations.

## C Question Generation Flows

This appendix details the per-type generation procedures. The family-level overview is in Section 3.3.

### C.1 Single-Document Flows

**Basic, Semantic, Intra-Document Reasoning,** and **Miscellaneous** questions each begin by sampling a single document and generating one question from it. All four follow the same pipeline—generate question, validate conformance, produce gold answer, extract atomic facts—but differ in sampling criteria, prompt guidance, and validation logic.

**Basic.** A random document is sampled and the LLM generates a retrieval-oriented question. The prompt steers toward moderate keyword overlap: enough that the question is clearly related to the document, but not so much that exact phrase matching trivially solves it. Key guidance includes avoiding long verbatim phrases from the source, avoiding complex multi-part questions, and using varied phrasing so that the question set does not collapse into a uniform “What is...” distribution. Few-shot examples of good and bad questions are provided to calibrate length and specificity.

**Semantic.** The generation pipeline is identical to basic questions, but the prompt guidance is substantially different. Where basic questions permit moderate keyword overlap, semantic questions actively suppress it: the LLM is instructed to avoid keyword matches where possible, limit qualifiers and scoping details, and produce “challenging, loose-match, semantic-type” queries. The few-shot examples are correspondingly harder, demonstrating roundabout phrasing that relies on meaning rather than surface terms.

**Intra-Document Reasoning.** This type targets a specific failure mode: systems that chunk documents and embed each chunk independently tend to miss questions whose answer requires combining information from distant sections of the same document. At the same time, expanding every retrieved document in full risks flooding the context window with irrelevant text. Intra-doc reasoning questions probe this tension directly. They also screen for a subtler failure: cases where one section of a document appears to contain the answer but a different section disqualifies it entirely.

During sampling, a minimum length threshold filters out documents too short to support cross-section questions. The LLM is guided to generate questions that require relating information from different parts of the document. A validation step then checks that the question cannot be answered from any single contiguous passage; questions that fail this check are rejected and regenerated.

**Miscellaneous.** These questions target the informal, off-topic files introduced during noise generation (e.g., files in `slack/memes` or `github/hackathons`). These documents sit outside the main scaffolding-driven generation and present a distinct retrieval challenge: they are topically peripheral and stored in less predictable locations, so systems tuned to the corpus’s dominant themes may overlook them. The generation flow is identical to basic questions—same prompt guidance, same validation—but restricted to the set of miscellaneous documents tracked in the generation cache.

## C.2 Multi-Document and Corpus-Exploration Flows

**Project Related, Constrained, High Level, and Info Not Found** questions require the generation model to explore the corpus before formulating a question. Each flow equips the LLM with file-system tools and provides context designed to guide exploration toward productive regions of the corpus.

**Project Related.** The model receives a project’s full document manifest and a read tool, then iteratively reads documents within the project until it identifies a question requiring information from multiple sources. The prompt steers the model toward four patterns:

- **Multi-document synthesis:** the answer requires combining parts from multiple documents to build a coherent picture.
- **Cross-cutting themes:** items mentioned incidentally across multiple documents but never the primary focus of any single one.
- **Contradictions:** conflicting information across documents, requiring the answer to acknowledge and reconcile the discrepancy.
- **Causal chains:** tracing a cause-and-effect relationship through multiple documents to construct a coherent narrative.

All documents read during exploration are tracked. After question generation, a separate step receives the question and all read documents and identifies the minimal subset necessary for a correct answer; this filtered set becomes the gold documents. Because these questions are broader than single-document types, they have a higher likelihood that documents outside the original project also contain relevant information, an inherent property of multi-document questions at this corpus scale that the quality assurance process is designed to address.

**Constrained.** A constrained question contains qualifiers that narrow the correct answer to a single document, even though many other documents in the corpus are superficially relevant. The qualifiers act as filters: each one eliminates documents that would otherwise match. This tests whether a retrieval system can distinguish surface-level relevance from true relevance under specific conditions: naive systems will return many partially matching documents, but only the document satisfying all constraints contains the correct answer.

Generation proceeds in two phases. In the exploration phase, an LLM agent equipped with file-system tools (`glob`, `grep`, `read`, etc.) performs the following:

1. Explores the directory structure to find clusters of topically related documents.
2. Reads documents within a cluster and identifies the axes along which they differ.
3. Formulates a question whose qualifiers narrow the answer to a single gold document, and identifies explicit distractor documents.

Previously explored document paths from prior constrained question runs are provided to prevent over-clustering in the same corpus regions. In the second phase, the gathered context is turned into a gold answer and facts. Unlike single-document types where facts are extracted from a separately produced answer, constrained questions generate the answer and facts jointly. The distractor documents are used to produce anti-hallucination facts: negative statements crafted to catch specific errors a system might make if it retrieves a distractor instead of the gold document (e.g., “The answer must not claim the Postgres database is the in-cluster StatefulSet, since the scenario specifies externally managed Postgres”).



**High Level.** These questions are answerable from the company overview and initiatives—the top-level scaffolding documents—but should not be answerable from any single document in the corpus. They test whether a system can synthesize broad organizational knowledge distributed across many documents. Generation proceeds in three stages:

1. **Candidate generation.** The LLM receives the company overview and initiatives and produces a batch of candidate queries. Guidance steers questions toward cross-cutting organizational patterns rather than point lookups, and requires varied phrasing and topic spread across the batch.
2. **Validation.** Each candidate is checked by a separate LLM agent equipped with file-system tools (glob, grep, read, etc.). The agent attempts to find a single document that directly answers the query; if it succeeds, the query is rejected as too specific.
3. **Answer and fact generation.** For each validated query, a gold answer is produced from the company overview and initiatives, and answer facts are extracted.

High-level questions carry no expected document identifiers, since the answer derives from organizational context distributed across the corpus and multiple different document subsets may each suffice. There is no strict guarantee that every question is fully answerable from the document set. The assumption is that across the full set of high-level questions, the relevant organizational context will typically exist somewhere in the corpus.

**Info Not Found.** An LLM agent equipped with file-system tools (glob, grep, read, etc.) explores the corpus to find a cluster of topically related documents. After reading several documents in the cluster, the agent identifies the dimensions along which they differ and crafts a natural-sounding query that is related to the cluster’s subject matter but not answerable from the documents. Corpus exploration is essential: by reading what the corpus actually contains, the model can deliberately adjust scoping, constraints, or details to fall just outside the available information, producing questions that are reliably unanswerable rather than generating blind queries. The query is designed so that surface-level keyword or topic matching would retrieve plausible-looking documents, but any answer derived from them would be hallucinated.

Previously explored document paths are tracked in a generation cache and provided in subsequent runs to prevent the agent from revisiting the same corpus regions. The gold answer and evaluation fact are predefined rather than generated: the expected answer states that the information is not available, and the single fact checks that the response acknowledges this rather than fabricating an answer.

### C.3 Structure-Dependent Flows

**Conflicting Info** and **Completeness** questions depend on document structures created during earlier pipeline stages rather than discovering them through exploration.

**Conflicting Info.** These questions target cases where one document supersedes another, typically when a newer document invalidates specific facts from an older one. This creates both a reasoning and a retrieval challenge: if the retriever returns only the superseded document, the system will report outdated information as fact. The generation model receives both documents from the near-duplicate pairs generated during the noise stage (Appendix B.4) and produces a question that requires reconciling the discrepancy: identifying which document supersedes the other and providing the correct, up-to-date answer. As with constrained questions, the answer and facts are generated jointly, and anti-hallucination facts are included to catch responses that report only the outdated information.

**Completeness.** These questions test whether a system can exhaustively retrieve all documents needed to answer a question, not just a relevant subset. Partial retrieval is insufficient: if even one required document is missing, aggregated counts will be wrong, lists will be incomplete, and comparisons will be skewed, making completeness a direct test of recall depth.

Questions are built from the tightly coupled document clusters generated during the high-fidelity phase (Appendix B.2), where 4–10 documents were produced with full mutual visibility and facts deliberately distributed so that no single document contains the complete answer. Because the target question already exists from the cluster generation step, this flow focuses on filtering: an LLM

Table 9: Corpus composition by source type. Lengths are measured in characters from the exported plain-text representation, sampled over 500 documents per source type.

| #            | Source Type  | Doc Count      | Mean Length | Median Length | P95 Length |
|--------------|--------------|----------------|-------------|---------------|------------|
| 1            | Slack        | 285,605        | 3,427       | 3,294         | 5,618      |
| 2            | Gmail        | 121,390        | 7,101       | 6,909         | 9,390      |
| 3            | Linear       | 35,308         | 5,059       | 4,875         | 7,529      |
| 4            | Google Drive | 25,108         | 7,160       | 7,094         | 10,113     |
| 5            | HubSpot      | 15,017         | 3,141       | 3,004         | 4,756      |
| 6            | Fireflies    | 10,173         | 11,605      | 11,074        | 16,315     |
| 7            | GitHub       | 8,052          | 4,741       | 4,664         | 6,910      |
| 8            | Jira         | 6,120          | 5,563       | 5,596         | 7,454      |
| 9            | Confluence   | 5,189          | 9,852       | 9,660         | 13,834     |
| <b>Total</b> |              | <b>511,962</b> |             |               |            |

Table 10: Number of gold documents per source type for each question category. High Level and Info Not Found carry no gold document identifiers by design. A single question may span multiple source types.

| Question Type    | Confl. | Fire. | GH | Gmail | GDrive | HS | Jira | Linear | Slack | Total Qs |
|------------------|--------|-------|----|-------|--------|----|------|--------|-------|----------|
| Basic            | 19     | 11    | 22 | 15    | 24     | 18 | 21   | 24     | 21    | 175      |
| Semantic         | 15     | 4     | 14 | 18    | 14     | 14 | 23   | 10     | 13    | 125      |
| Intra-Doc Reas.  | 2      | 3     | 2  | 9     | 0      | 1  | 2    | 9      | 12    | 40       |
| Project Related  | 30     | 0     | 18 | 7     | 7      | 0  | 29   | 9      | 14    | 40       |
| Constrained      | 16     | 0     | 0  | 0     | 1      | 0  | 13   | 2      | 3     | 30       |
| Conflicting Info | 14     | 2     | 2  | 2     | 8      | 1  | 4    | 4      | 1     | 20       |
| Completeness     | 16     | 3     | 1  | 3     | 2      | 0  | 5    | 0      | 5     | 20       |
| Miscellaneous    | 3      | 2     | 1  | 0     | 3      | 0  | 2    | 0      | 9     | 20       |
| High Level       |        |       |    |       | —      |    |      |        |       | 10       |
| Info Not Found   |        |       |    |       | —      |    |      |        |       | 20       |

evaluates each document’s contribution and removes any that prove unnecessary in practice. Clusters that reduce to fewer than two required documents after filtering are discarded. The remaining set defines the gold documents, and answer facts are extracted as individually verifiable claims to enable fine-grained recall measurement.

## D Dataset Statistics

This appendix provides exact corpus and question-level statistics for the released dataset.

### D.1 Corpus Composition

Table 9 provides exact document counts and length statistics by source type.

The variation is substantial: meeting transcripts (Fireflies) and wiki pages (Confluence) are 3–4× longer than Slack threads and CRM records (HubSpot), reflecting the different information density and formatting conventions of each source type.

Slack threads are notably longer than typical real-world threads, where many are short or even single-message. This is discussed further in Section 6.

### D.2 Source Type Coverage Across Question Types

Table 10 shows how often each source type appears as a gold document across question categories. A single question may span multiple source types, so row totals can exceed the question count.

Two patterns stand out. Confluence and Jira are substantially overrepresented as gold sources relative to their corpus share (1.0% and 1.2% of documents, respectively, but appearing in 24% and 21% of questions with gold documents). This reflects the nature of structured documentation: wiki pages and

Table 11: Gold document set size statistics by question type. High Level and Info Not Found carry no expected documents.

| Question Type            | Mean | Median | Min | Max |
|--------------------------|------|--------|-----|-----|
| Basic                    | 1.0  | 1      | 1   | 1   |
| Semantic                 | 1.0  | 1      | 1   | 1   |
| Intra-Document Reasoning | 1.0  | 1      | 1   | 1   |
| Miscellaneous            | 1.0  | 1      | 1   | 1   |
| Conflicting Info         | 2.0  | 2      | 2   | 2   |
| Constrained              | 1.4  | 1      | 1   | 2   |
| Project Related          | 4.2  | 3.5    | 2   | 9   |
| Completeness             | 6.5  | 6      | 2   | 10  |
| High Level               | —    | —      | —   | —   |
| Info Not Found           | —    | —      | —   | —   |

tickets are more likely to contain definitive, self-contained answers than conversational messages. Conversely, Slack constitutes 56% of the corpus but appears as a gold source in only 17% of questions.

### D.3 Gold Document Set Size by Question Type

Table 11 reports the number of gold documents per question. Single-document types (Basic, Semantic, Intra-Document Reasoning, Miscellaneous) have exactly one gold document by construction. Conflicting Info always pairs two documents. Constrained questions have 1–2 gold documents, as the count is determined by the LLM based on query complexity. Project Related and Completeness questions require the most documents, with Completeness reaching up to 10.